Detecting Phishing Websites Using Neural Networks

Midterm Progress Report

Liliana Matte, Whitney Akred, Leiton Peterson
COSC 50523: Deep Learning
Path A: Applied Deep Learning

I. Problem and Changes Since the Proposal

Phishing websites imitate legitimate websites to trick users into revealing sensitive information. Because phishing websites can change rapidly, automated detection is an important cybersecurity problem [2]. Our task is to create a binary classification: predict whether a website is *phishing* (1) or *legitimate* (0). Missing a phishing site (a false negative) is the more harmful error, so recall on the phishing class is our primary metric.

The proposal compared an MLP with a Logistic Regression baseline on the dataset's extracted features. The feedback asked us to define a baseline and at least two more models, justify whether these model types suit the data, explain how URLs are converted to numbers, and differ from another team using the same dataset. Our first experiments (Section V) show that the full feature set is too easy to answer the model-choice question, so the rest of the work focuses on a challenging but more realistic question: *when only the URL is available, does a neural network that reads the raw URL outperform models built on hand-crafted URL features?*

II. Dataset

We use the PhiUSIIL Phishing URL Dataset from the UCI Machine Learning Repository [1]. It contains 235,795 websites and 56 columns: 50 numeric features, 5 text columns, and a label. There are no missing values and no duplicate rows. In the original file, label 1 means legitimate; we inverted the label so that phishing is the positive class (1). **Link:** https://archive.ics.uci.edu/dataset/967/phiusiil+phishing+url+website

TABLE I
Dataset Summary

	Websites	Share
Legitimate (0)	134,850	57.19%
Phishing (1)	100,945	42.81%
Training split (70%)	165,056	
Validation split (15%)	35,369	
Test split (15%)	35,370	

III. Preprocessing and How URLs Become Numbers

Models can only learn from numbers, so **we removed all text (non-numeric) columns** before training: FILENAME, URL, Domain, TLD, and Title. Every model therefore uses only the 50 numeric features that the dataset authors already extracted from each URL and webpage instead of the text labels. These features fall into two groups:

- **URL features**, computed from the URL string alone: URL and domain length, number of subdomains, counts and ratios of letters, digits, and special characters, use of HTTPS, whether the domain is an IP address, obfuscation counts, and URLSimilarityIndex.
- **Page features**, which require loading the page: lines of code, number of images, scripts, and links, title match scores, and presence of forms or password fields.

We used a 70/15/15 train/validation/test split with a fixed random seed (42), which keeps the 42.81% phishing rate in every split. We standardized all numerical features using only information from the training dataset to ensure a fair evaluation of the model on unseen data. All comparisons below use the validation split.

Leakage check. Because the duplicate check included the URL text, two different sites could become identical once the text columns were removed. We found that only 151 of 35,369 validation rows (0.43%) match exactly a training row, which is too small to explain the high scores below.

IV. Models

All models use the same inputs and splits and are scored by the same evaluation function (accuracy, precision, recall, F1, and confusion-matrix counts).

- **Baseline: Logistic Regression.** A linear model that weights each feature and converts the weighted sum to a phishing probability (scikit-learn, L2 penalty, $C = 1.0$, lbfgs solver). It converged in 13 iterations.
- **Random Forest.** 100 decision trees (seed 42), a strong classical model for tabular data that tests whether or not a nonlinear model is needed.
- **MLP.** A fully connected network built in Keras with hidden layers of ReLU and a sigmoid output, trained with binary cross-entropy and Adam (learning rate 0.001), batch size 256, for 20 epochs. The main model uses hidden layers of 64 and 32 units (5,377 parameters). As a controlled experiment, we only changed the network size: [32], [64, 32], and [128, 64, 32].

V. Preliminary Results

The full feature set is almost perfectly separable. Logistic Regression missed only 8 of 15,142 phishing sites, and the Random Forest classified every validation site correctly. The

TABLE II
VALIDATION RESULTS ("MISSED" = PHISHING SITES LABELED LEGITIMATE; "FALSE ALARMS" = LEGITIMATE SITES LABELED PHISHING.)

Model (validation split)	Accuracy	Precision	Recall	Missed	False alarms
Logistic Regression, all features	0.9998	1.0000	0.9995	8	0
LR, URLSimilarityIndex only	0.9963	1.0000	0.9913	131	0
LR, without URLSimilarityIndex	0.9990	0.9993	0.9985	23	11
Random Forest	1.0000	1.0000	1.0000	0	0
MLP [32]	0.9999	1.0000	0.9999	2	0
MLP [64, 32]	0.9999	1.0000	0.9998	3	0
MLP [128, 64, 32]	0.9999	1.0000	0.9998	3	0

baseline's largest weights are sensible: a high URLSimilarityIndex (-6.88) and HTTPS (-3.83) push toward legitimate, while a higher ratio of special characters ($+1.84$) or letters ($+1.77$) in the URL pushes toward phishing.

The result does not depend on one feature. URLSimilarityIndex is the most important feature for both Logistic Regression and the Random Forest (importance 0.186, followed by NoOfExternalRef at 0.169 and LineOfCode at 0.149). It reaches 99.63% accuracy on its own, but removing it still leaves 99.90%, so the dataset is easy across many features, not because of a single shortcut.

MLP training. The [64, 32] MLP reached 99.97% validation accuracy after one epoch. Validation loss was lowest around epoch 9 and then rose while training loss kept falling, a mild sign of overfitting that early stopping should fix.

MLP size experiment. Growing the network from 1,665 parameters ([32]) to 16,897 ([128, 64, 32]) did not improve the results: the three networks missed 2, 3, and 3 of 15,142 phishing sites, and none raised a false alarm. A difference of one error is within noise, so a single 32-unit layer is enough for these features. The MLPs landed between Logistic Regression (8 missed) and the Random Forest (0 missed), but these gaps amount to only a few sites.

Answer to the model-choice question. With these 50 features, a simple linear model already reaches the ceiling, so neither the Random Forest nor the MLP can show an advantage in data when compared. A neural network is not justified for this input, which is why we are changing the setup for the rest of the project.

VI. DIFFICULTIES AND LIMITATIONS

- **Ceiling effect.** With every model above 99.9%, a difference of a few errors is not meaningful, so the MLP size experiment cannot separate the three networks.
- **Realism.** Page features require downloading a suspicious page, which a user may want to avoid. Results that use them may overstate what is possible at the moment a link is clicked.
- **Same domain in several splits.** A random split can place pages from the same domain in both training and validation, so scores may partly reflect memorized domains.
- **Evaluation consistency.** Our first MLP size experiment was scored on the test split; we reran it on validation so all models are compared on the same data and the test set stays untouched until the end.

VII. REMAINING WORK

To make the comparison fair, the rest of the project uses **only information available from the URL**. Because URLSimilarityIndex alone reaches 99.63% accuracy, we will first check whether URL-only features are still at the ceiling, with and without it. All models will use the same rows and splits:

1) **Baseline:** Logistic Regression on the URL features only (all page features removed).
2) **Random Forest** on the URL features only.
3) **MLP** on the URL features only.
4) **Character-level CNN on the raw URL**. Instead of using hand-crafted features, each character of the URL is converted to a number, the sequence is padded to a fixed length, and the network learns its own representation through an embedding layer, 1D convolutions, and pooling. This is our answer to how URLs are converted to numbers.

Controlled experiment: hand-crafted URL features (Models 1-3) vs. the learned character encoding (Model 4), with the data and splits held fixed. We will also examine representative misclassified URLs. **If time allows:** a split in which no domain appears in more than one split, and one CNN design choice (kernel sizes).

Timeline: URL-only baselines and MLP by Nov. 1; character-level CNN by Nov. 15; error analysis and optional experiments by Nov. 22; final report and reproducible code by Dec. 2.

Code: https://github.com/lilianam37/deep-minds

Use of AI tools: We used Claude to brainstorm and compare candidate models for our dataset's numeric tabular features, which led us to consider Random Forest alongside Logistic Regression and the MLP. We also used it to research how character-level CNNs process raw URLs and to consider whether a CNN would be a useful next step for our project. Finally, it helped generate LaTeX syntax for formatting the dataset summary and results tables based off our Google Colab graphs and numberical values in testing. The team independently selected the experiments, implemented and evaluated the models, and verified the reported results and conclusions against our Google Colab notebooks.

REFERENCES

[1] A. Prasad and S. Chandra, "PhiUSIIL: A diverse security profile empowered phishing URL detection framework based on similarity index and incremental learning," *Computers & Security*, vol. 136, Art. no. 103545, Jan. 2024.

[2] H. Ghalechyan, E. Israyelyan, A. Arakelyan, G. Hovhannisyan, and A. Davtyan, "Phishing URL detection with neural networks: an empirical study," *Scientific Reports*, vol. 14, Art. no. 25134, Oct. 2024.

[3] A. Rawla, S. Singh, M. Daniyal, and P. Dubey, "Detection of phishing attacks in PhiUSIIL dataset using deep learning," *Procedia Computer Science*, vol. 259, pp. 543–552, 2025.